Celery, Redis & Flower — Complete Notes

1. What is Celery?

Celery is an open-source distributed task queue for Python. It is used to execute background and scheduled tasks asynchronously.

Common use cases:

- Sending emails and OTPs
- Sending notifications
- Generating PDF reports and invoices
- Processing uploaded images
- Processing payments
- Importing large CSV files
- Running scheduled jobs

2. What is a Background Task?

A background task runs independently of the main HTTP request-response cycle.

Without Celery:

```
User → Django API → Send Email → Generate PDF → Response
```

The user waits until all operations finish.

With Celery:

```
User → Django API → Redis Broker → Celery Worker
          |
          └── Return response immediately
```

The worker processes the task in the background.

3. What is Redis?

Redis is an in-memory data store that supports different data structures.

It can be used as:

- Cache
- Message broker
- Result backend
- Session storage
- Temporary data store

Redis as a Message Broker

A broker receives task messages from the application and delivers them to workers.

```
Django Application → Redis → Celery Worker
```

Redis as a Result Backend

A result backend stores task states and return values.

```
Celery Worker → Redis Result Backend → Application
```

Example configuration:

```
CELERY_BROKER_URL = "redis://localhost:6379/0"
CELERY_RESULT_BACKEND = "redis://localhost:6379/1"
```

Using separate Redis database numbers is a convention, not a requirement.

4. What is Flower?

Flower is a web-based monitoring tool for Celery.

It provides:

- Worker monitoring
- Active task monitoring
- Completed and failed task information
- Task execution time
- Worker status
- Task history
- Task revocation

Default dashboard:

```
http://localhost:5555
```

Flower monitors Celery. It does not execute tasks.

5. Celery Architecture

The main components are:

1. **Producer:** Sends tasks.
2. **Broker:** Delivers task messages.
3. **Worker:** Executes tasks.
4. **Result Backend:** Stores task states and results.
5. **Celery Beat:** Schedules periodic tasks.
6. **Flower:** Monitors workers and tasks.

```
                  Django / FastAPI
                      Producer
                         |
                         v
                    Redis Broker
                         |
                         v
                   Celery Worker
                         |
                         v
                    Execute Task
                         |
                         v
                   Result Backend

                   Flower Dashboard
                     Monitors tasks

                     Celery Beat
                  Schedules tasks
```

6. Installation

Install the required packages:

```
pip install django celery redis flower
```

Check Redis:

```
redis-cli ping
```

Expected output:

```
PONG
```

7. Django Project Structure

```
myproject/
│
├── manage.py
│
├── myproject/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   └── celery.py
│
└── accounts/
    ├── __init__.py
    ├── models.py
    ├── views.py
    └── tasks.py
```

8. Configure Celery with Django

Step 1: Create celery.py

Inside the main Django project directory:
`myproject/celery.py`

```
import os

from celery import Celery

os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "myproject.settings"
)

app = Celery("myproject")

app.config_from_object(
    "django.conf:settings",
    namespace="CELERY"
)

app.autodiscover_tasks()
```

Explanation:

- `Celery("myproject")` creates the Celery application.
- `config_from_object()` loads configuration from Django settings.
- `autodiscover_tasks()` discovers tasks.py files in installed Django applications.

Step 2: Update init.py

Inside `myproject/__init__.py`:

```
from .celery import app as celery_app

__all__ = ("celery_app",)
```

This ensures the Celery application is loaded when the Django project is imported.

Step 3: Configure settings.py

```
CELERY_BROKER_URL = "redis://localhost:6379/0"

CELERY_RESULT_BACKEND = "redis://localhost:6379/1"

CELERY_ACCEPT_CONTENT = ["json"]

CELERY_TASK_SERIALIZER = "json"

CELERY_RESULT_SERIALIZER = "json"

CELERY_TIMEZONE = "Asia/Dhaka"

CELERY_ENABLE_UTC = True

CELERY_RESULT_EXPIRES = 3600
```

For production, read connection URLs and credentials from environment variables.

9. Create Celery Tasks

Create `accounts/tasks.py`:

```
from celery import shared_task


@shared_task
def add_numbers(a, b):
    return a + b
```

Email Task Example

```
from celery import shared_task
from django.core.mail import send_mail


@shared_task
def send_welcome_email(email):
    send_mail(
        subject="Welcome!",
        message="Welcome to our platform.",
        from_email="admin@example.com",
        recipient_list=[email],
        fail_silently=False,
    )

    return "Email sent successfully"
```

10. Run Celery Worker

Open a separate terminal:

```
celery -A myproject worker --loglevel=info
```

For local Windows development:

```
celery -A myproject worker --loglevel=info --pool=solo
```

The Django server and Celery worker run as separate processes.

11. Calling Celery Tasks

Direct Function Call

```
add_numbers(10, 20)
```

This executes synchronously in the current process.

Using delay()

```
add_numbers.delay(10, 20)
```

This submits the task asynchronously.

Using apply_async()

```
add_numbers.apply_async(
    args=[10, 20],
    countdown=10
)
```

`delay()` is a shortcut. `apply_async()` supports additional execution options.

delay()	apply_async()
Simple interface	More configurable
Passes task arguments	Passes arguments and options
Less flexible	Supports countdown, ETA, queue, expiration

12. AsyncResult

`AsyncResult` represents an asynchronous task execution.

```
result = add_numbers.delay(10, 20)

print(result.id)
print(result.status)
print(result.ready())
```

Check success:

```
if result.successful():
    print("Task completed successfully")
```

Get the result:

```
value = result.get(timeout=10)

print(value)
```

Expected result:

```
30
```

Important: `result.get()` can block the current process while waiting. Avoid using it inside normal API request handling unless blocking is intentional.

13. Celery Task States

State	Meaning
PENDING	State is unknown or not yet stored
RECEIVED	Worker received the task
STARTED	Task started execution
SUCCESS	Task completed successfully
FAILURE	Task failed
RETRY	Task will be retried

REVOKED	Task was revoked

PENDING does not always mean the task is waiting in the queue. It can also mean that no state has been stored.

14. Retry Mechanism

Retries are useful when temporary failures occur, such as network errors or unavailable external services.

```
from celery import shared_task


@shared_task(
    bind=True,
    max_retries=3,
    default_retry_delay=10
)
def send_email(self, email):
    try:
        # Email sending logic
        print(f"Sending email to {email}")

        return "Email sent"

    except Exception as exc:
        raise self.retry(exc=exc)
```

Explanation

- `bind=True` : Gives the task access to its task instance through `self` .
- `max_retries=3` : Allows up to three retries.
- `default_retry_delay=10` : Sets the default retry delay in seconds.
- `self.retry()` : Requests another execution attempt.

Exponential Backoff

```
@shared_task(
    bind=True,
    autoretry_for=(ConnectionError,),
    retry_backoff=True,
    retry_backoff_max=600,
    retry_jitter=True,
    max_retries=5,
)
def call_external_api(self):
    # External API request
    pass
```

Exponential backoff increases the delay between retries. Jitter adds randomness to reduce simultaneous retry spikes.

15. Celery Beat — Periodic Tasks

Celery Beat is a scheduler that sends tasks to the broker at configured times or intervals.

Examples:

- Daily reports
- Monthly invoices
- Expired session cleanup
- Reminder emails
- Scheduled notifications

Step 1: Create a Task

```
from celery import shared_task



@shared_task
def daily_report():
    print("Daily report generated")
```

Step 2: Configure the Schedule

Inside `settings.py`:

```
from celery.schedules import crontab



CELERY_BEAT_SCHEDULE = {
    "daily-report": {
        "task": "accounts.tasks.daily_report",
        "schedule": crontab(
            hour=9,
            minute=0
        ),
    },
}
```

This schedules the task every day at 9:00 AM in the configured timezone.

Common Schedule Examples

Every 30 seconds:

```
"schedule": 30.0
```

Every 5 minutes:

```
"schedule": 300.0
```

Every hour:

```
"schedule": 3600.0
```

Every day at midnight:

```
"schedule": crontab(hour=0, minute=0)
```

Every Monday at 9 AM:

```
"schedule": crontab(
    day_of_week=1,
    hour=9,
```

```
    minute=0
)
```

Step 3: Run Celery Beat

```
celery -A myproject beat --loglevel=info
```

Step 4: Run Celery Worker

```
celery -A myproject worker --loglevel=info
```

Run Beat as a separate service in production. Avoid running multiple Beat schedulers for the same schedule.

16. Task Execution Options

countdown

Execute after a delay:

```
send_welcome_email.apply_async(
    args=["user@example.com"],
    countdown=30
)
```

eta

Schedule execution at a specific datetime:

```
from datetime import datetime, timedelta, timezone

scheduled_time = (
    datetime.now(timezone.utc)
    + timedelta(minutes=5)
)

send_welcome_email.apply_async(
    args=["user@example.com"],
    eta=scheduled_time
)
```

queue

Send a task to a specific queue:

```
generate_report.apply_async(
    args=[10],
    queue="reports"
)
```

expires

Set task expiration:

```
send_welcome_email.apply_async(
    args=["user@example.com"],
    expires=60
)
```

Long countdown and ETA tasks need special consideration when using Redis as a broker. For large-scale scheduling, Celery Beat or a dedicated scheduling system is often more appropriate.

17. Groups, Chains and Chords

Celery provides primitives for coordinating multiple tasks.

Group

A group runs multiple tasks independently.

```
from celery import group

job = group(
    add_numbers.s(10, 20),
    add_numbers.s(30, 40),
    add_numbers.s(50, 60),
)

result = job.apply_async()
```

Chain

A chain executes tasks sequentially. The result of one task is passed to the next task.

```
from celery import chain

job = chain(
    add_numbers.s(10, 20),
    multiply_number.s(2),
)

result = job.apply_async()
```

Task definitions:

```
@shared_task
def add_numbers(a, b):
    return a + b


@shared_task
def multiply_number(value, multiplier):
    return value * multiplier
```

Chord

A chord executes a group of tasks and then runs a callback after the group completes.

```
from celery import chord

job = chord(
    [
        add_numbers.s(10, 20),
        add_numbers.s(30, 40),
    ],
    calculate_total.s()
)

result = job.apply_async()
```

Callback:

```
@shared_task
def calculate_total(results):
    return sum(results)
```

Primitive	Purpose
Group	Run independent tasks
Chain	Run tasks sequentially
Chord	Run a group, then a callback
Signature	Represents a task invocation

Chord support depends on the configured result backend.

18. Multiple Queues

Multiple queues help separate different workloads.

Example queues:

```
default
emails
reports
payments
notifications
```

Configure queues:

```
from kombu import Queue


CELERY_TASK_QUEUES = (
    Queue("default"),
    Queue("emails"),
    Queue("reports"),
)
```

Route tasks:

```
CELERY_TASK_ROUTES = {
    "accounts.tasks.send_welcome_email": {
        "queue": "emails"
```

```
    },

    "reports.tasks.generate_report": {
        "queue": "reports"
    },
}
```

Run email worker:

```
celery -A myproject worker -Q emails --loglevel=info
```

Run report worker:

```
celery -A myproject worker -Q reports --loglevel=info
```

19. Flower Monitoring

Install Flower:

```
pip install flower
```

Start Flower:

```
celery -A myproject flower
```

Open:

```
http://localhost:5555
```

Custom port:

```
celery -A myproject flower --port=5556
```

Enable task events:

```
celery -A myproject worker --loglevel=info --events
```

You can also configure:

```
CELERY_WORKER_SEND_TASK_EVENTS = True
CELERY_TASK_SEND_SENT_EVENT = True
```

Flower authentication:

```
celery -A myproject flower --basic_auth=username:strong-password
```

Protect Flower using authentication, HTTPS, and restricted network access. Do not expose an unsecured monitoring dashboard publicly.

20. Celery with FastAPI

Celery can also be used with FastAPI.

Project Structure

```
fastapi_project/
├── main.py
├── celery_app.py
└── tasks.py
```

celery_app.py

```
from celery import Celery

celery_app = Celery(
    "fastapi_project",
    broker="redis://localhost:6379/0",
    backend="redis://localhost:6379/1",
)

celery_app.conf.update(
    task_serializer="json",
    accept_content=["json"],
    result_serializer="json",
    timezone="Asia/Dhaka",
    enable_utc=True,
)
```

tasks.py

```
from celery_app import celery_app


@celery_app.task
def add_numbers(a, b):
    return a + b
```

main.py

```
from fastapi import FastAPI

from tasks import add_numbers

app = FastAPI()


@app.post("/calculate")
def calculate(a: int, b: int):
    task = add_numbers.delay(a, b)

    return {
        "task_id": task.id,
        "status": "Task submitted"
    }
```

Run FastAPI:

```
uvicorn main:app --reload
```

Run worker:

```
celery -A celery_app.celery_app worker --loglevel=info
```

Run Flower:

```
celery -A celery_app.celery_app flower
```

21. Redis Commands

Connect to Redis:

```
redis-cli
```

Test connection:

```
PING
```

Set a value:

```
SET name Faruk
```

Get a value:

```
GET name
```

Delete a key:

```
DEL name
```

Check whether a key exists:

```
EXISTS name
```

Set expiration:

```
SET otp 123456 EX 60
```

Check remaining expiration:

```
TTL otp
```

Get server information:

```
INFO
```

Scan keys:

```
SCAN 0
```

Avoid `KEYS *` on production Redis instances with large datasets because it can block the server.

22. Docker Compose Setup

Example `docker-compose.yml`:

```
services:
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data

  web:
    build: .
    command: python manage.py runserver 0.0.0.0:8000
    ports:
      - "8000:8000"
    environment:
      CELERY_BROKER_URL: redis://redis:6379/0
```

```
      CELERY_RESULT_BACKEND: redis://redis:6379/1
    depends_on:
      - redis

  celery_worker:
    build: .
    command: celery -A myproject worker --loglevel=info
    environment:
      CELERY_BROKER_URL: redis://redis:6379/0
      CELERY_RESULT_BACKEND: redis://redis:6379/1
    depends_on:
      - redis

  celery_beat:
    build: .
    command: celery -A myproject beat --loglevel=info
    environment:
      CELERY_BROKER_URL: redis://redis:6379/0
      CELERY_RESULT_BACKEND: redis://redis:6379/1
    depends_on:
      - redis

  flower:
    build: .
    command: celery -A myproject flower --port=5555
    ports:
      - "5555:5555"
    environment:
      CELERY_BROKER_URL: redis://redis:6379/0
      CELERY_RESULT_BACKEND: redis://redis:6379/1
    depends_on:
      - redis

volumes:
  redis_data:
```

Start services:

```
docker compose up --build
```

Stop services:

```
docker compose down
```

View worker logs:

```
docker compose logs -f celery_worker
```

Important: Inside Docker Compose, use the Redis service name:

```
redis://redis:6379/0
```

Do not use `localhost` to connect to another container. Also, `depends_on` does not guarantee that Redis is ready to accept connections. Use health checks and retry logic for robust deployments.

23. Production Best Practices

1. Use Environment Variables

```
import os

CELERY_BROKER_URL = os.environ["CELERY_BROKER_URL"]

CELERY_RESULT_BACKEND = os.environ.get(
    "CELERY_RESULT_BACKEND"
)
```

2. Configure Time Limits

```
CELERY_TASK_SOFT_TIME_LIMIT = 300
CELERY_TASK_TIME_LIMIT = 360
```

3. Configure Worker Concurrency

```
celery -A myproject worker --concurrency=4 --loglevel=info
```

Choose concurrency according to workload, CPU, memory, and worker pool.

4. Configure Late Acknowledgement Carefully

```
CELERY_TASK_ACKS_LATE = True
```

Late acknowledgement can improve recovery behavior in some failure scenarios, but duplicate execution is possible.

5. Make Tasks Idempotent

An idempotent task can be executed more than once without causing unintended duplicate effects.
Problematic example:

```
@shared_task
def add_balance(user_id, amount):
    user = User.objects.get(id=user_id)
    user.balance += amount
    user.save()
```

If this task executes twice, the balance may increase twice.
Better approaches:

- Use unique transaction IDs.
- Track processed operations.
- Use database uniqueness constraints.
- Use database transactions.
- Use idempotency keys for supported external APIs.

6. Use transaction.on_commit()

When a task depends on data created inside a database transaction, submit it after the transaction commits.

```
from django.db import transaction

transaction.on_commit(
    lambda: send_welcome_email.delay(user.email)
)
```

This prevents a worker from trying to read a database record before it is committed.

7. Additional Recommendations

- Keep tasks small and focused.
- Handle exceptions properly.
- Configure retries carefully.
- Monitor worker health.
- Separate queues for different workloads.
- Protect Flower.
- Avoid storing large files in the result backend.
- Use structured logging.
- Monitor Redis memory and queue length.
- Configure Redis persistence according to requirements.
- Avoid running multiple Beat schedulers for the same schedule.

24. Common Errors and Solutions

Redis Connection Error

Possible causes:

- Redis is not running.
- Incorrect broker URL.
- Wrong port.
- Docker networking issue.
- Authentication issue.

Test:

```
redis-cli ping
```

Unregistered Task

Possible causes:

- Incorrect task name.
- Django app missing from `INSTALLED_APPS`.
- Task discovery issue.
- Worker not restarted after adding tasks.

Task Not Executing

Check:

- Redis connection.
- Worker status.
- Queue routing.
- Task registration.
- Worker logs.

Flower Cannot Connect

Check:

- Redis connection.
- Broker URL.
- Celery application name.
- Worker event configuration.

Task Always Shows PENDING

Possible causes:

- Task has not executed.
- Result backend is not configured.
- Result has expired.
- Incorrect task ID.
- State is not stored.

Remember: PENDING means the state is unknown or unavailable. It does not always mean the task is waiting in the queue.

Duplicate Task Execution

Possible causes:

- Message redelivery.
- Worker crash.
- Late acknowledgement.
- Retry configuration.
- Duplicate API requests.

Solution: design tasks to be idempotent.

25. Real-World E-commerce Workflow

When a customer places an order:

```
Customer
   |
   v
Django API
   |
   v
Save Order
   |
   v
Submit Celery Tasks
   |
   +-------------------+
   |                   |
   v                   v
Send Email         Generate Invoice
   |                   |
   v                   v
Notification       Save PDF
```

Possible background tasks:

- Send order confirmation.
- Generate invoice.
- Notify warehouse.
- Send shipping notification.
- Update inventory.
- Generate sales reports.

Payment processing requires special care. Submitting a background task does not mean payment succeeded. Verify payment with the payment provider and maintain reliable transaction records.

26. Interview Questions and Answers

Q1. What is Celery?

Celery is a distributed task queue for Python that executes tasks asynchronously or periodically.

Q2. Why do we use Celery?

To move time-consuming operations out of the main request-response cycle.

Q3. What is Redis?

Redis is an in-memory data store that can be used as a Celery broker and result backend.

Q4. What is a message broker?

A message broker receives messages from producers and delivers them to consumers.

Q5. What is a Celery worker?

A worker is a process that receives tasks from the broker and executes them.

Q6. What is Flower?

Flower is a web-based monitoring dashboard for Celery.

Q7. What is Celery Beat?

Celery Beat schedules periodic tasks.

Q8. What is the difference between delay() and apply_async()?

`delay()` is a simple shortcut. `apply_async()` provides more execution options.

Q9. What is AsyncResult?

AsyncResult represents an asynchronous task execution and provides its ID, state, and result.

Q10. What is a retry?

A retry schedules a failed task for another execution attempt.

Q11. What is the difference between group and chain?

A group runs tasks independently. A chain runs tasks sequentially.

Q12. What is a chord?

A chord runs a group of tasks and executes a callback after the group finishes.

Q13. Can Celery work without Redis?

Yes. RabbitMQ and other supported brokers can be used.

Q14. Can Celery work with FastAPI?

Yes. FastAPI can submit Celery tasks using `.delay()` or `.apply_async()`.

Q15. What is idempotency?

Idempotency means repeated execution produces the same intended final effect as a single execution.

Q16. What happens if a worker crashes?

Depending on acknowledgement settings and broker behavior, a task may be redelivered or require recovery. Tasks should safely handle retries and duplicates.

Q17. What is the difference between Celery and Celery Beat?

Celery executes tasks. Celery Beat schedules periodic tasks.

Q18. Should we use result.get() inside an API endpoint?

Usually not. It can block the request. Return a task ID and provide another endpoint to check status.

Q19. Redis vs RabbitMQ?

Redis is an in-memory data store that can act as a broker. RabbitMQ is a dedicated message broker with advanced messaging and routing capabilities.

Q20. Does Celery guarantee exactly-once execution?

No. Distributed task execution can involve retries and duplicate deliveries. Design tasks to be idempotent.

27. Practice Projects

Beginner

1. Build a Django project with Celery and Redis.
2. Create an addition task.
3. Send a welcome email asynchronously.
4. Return a task ID from an API.
5. Check task status.

Intermediate

1. Build an email notification system.
2. Implement task retries.
3. Schedule daily reports using Celery Beat.
4. Add Flower monitoring.
5. Create multiple queues.

Advanced

1. Build an e-commerce order processing system.
2. Generate invoices asynchronously.
3. Implement payment notifications.
4. Implement task idempotency.
5. Deploy using Docker Compose.
6. Handle worker crashes and retries.
7. Implement monitoring and logging.

28. Important Commands

Start Redis:

```
redis-server
```

Test Redis:

```
redis-cli ping
```

Start Django:

```
python manage.py runserver
```

Start Celery worker:

```
celery -A myproject worker --loglevel=info
```

Windows development worker:

```
celery -A myproject worker --loglevel=info --pool=solo
```

Start Celery Beat:

```
celery -A myproject beat --loglevel=info
```

Start Flower:

```
celery -A myproject flower
```

Start worker with events:

```
celery -A myproject worker --loglevel=info --events
```

Inspect active workers:

```
celery -A myproject inspect active
```

Inspect registered tasks:

```
celery -A myproject inspect registered
```

Inspect scheduled tasks:

```
celery -A myproject inspect scheduled
```

Inspect active queues:

```
celery -A myproject inspect active_queues
```

Revoke a task:

```
celery -A myproject control revoke TASK_ID
```

Revoking a task does not necessarily terminate one that is already running.

29. Important Concepts Summary

Concept	Meaning
Celery	Distributed task queue
Redis	In-memory data store and possible broker
RabbitMQ	Dedicated message broker
Producer	Sends tasks
Broker	Delivers task messages
Worker	Executes tasks

Result Backend	Stores task states and results
Flower	Monitoring dashboard
Celery Beat	Periodic task scheduler
Task	Unit of work
delay()	Simple asynchronous submission
apply_async()	Configurable asynchronous submission
AsyncResult	Represents task execution result
Retry	Re-executes a failed task
Group	Independent tasks
Chain	Sequential tasks

Chord	Group followed by callback
Queue	Holds messages waiting for workers
Idempotency	Makes repeated execution safe

30. Official Documentation

- Celery: https://docs.celeryq.dev/
- Celery Introduction: https://docs.celeryq.dev/en/stable/getting-started/introduction.html
- Celery with Django: https://docs.celeryq.dev/en/stable/django/first-steps-with-django.html
- Celery Tasks: https://docs.celeryq.dev/en/stable/userguide/tasks.html
- Celery Workers: https://docs.celeryq.dev/en/stable/userguide/workers.html
- Celery Periodic Tasks: https://docs.celeryq.dev/en/stable/userguide/periodic-tasks.html
- Celery Canvas: https://docs.celeryq.dev/en/stable/userguide/canvas.html
- Redis: https://redis.io/docs/latest/
- Flower: https://flower.readthedocs.io/

Final Learning Roadmap

Learn these topics in order:

1. Background task concepts
2. Celery architecture
3. Redis broker and result backend
4. Celery installation
5. Django integration
6. Creating and calling tasks
7. Task states and AsyncResult
8. Retry and error handling
9. Celery Beat
10. Groups, Chains, and Chords
11. Multiple queues
12. Flower monitoring
13. Docker deployment
14. Idempotency and production reliability

Final goal: Build a Django REST API that processes email notifications, invoice generation, and scheduled reports using Celery, Redis, and Flower.